DEVOPS ENGINEER ASSESSMENT

CI/CD Automation for .NET Deployment to Minikube

Azure DevOps · Docker · Kubernetes · Prometheus & Grafana

github.com/Ge0rak/minikube-cicd

AGENDA

What this presentation covers

1

Architecture

How the pieces fit together end to end

2

Core Tasks

Azure DevOps, Docker, Kubernetes, Monitoring

3

Bonus Work

Probes, autoscaling, layered testing, failover design

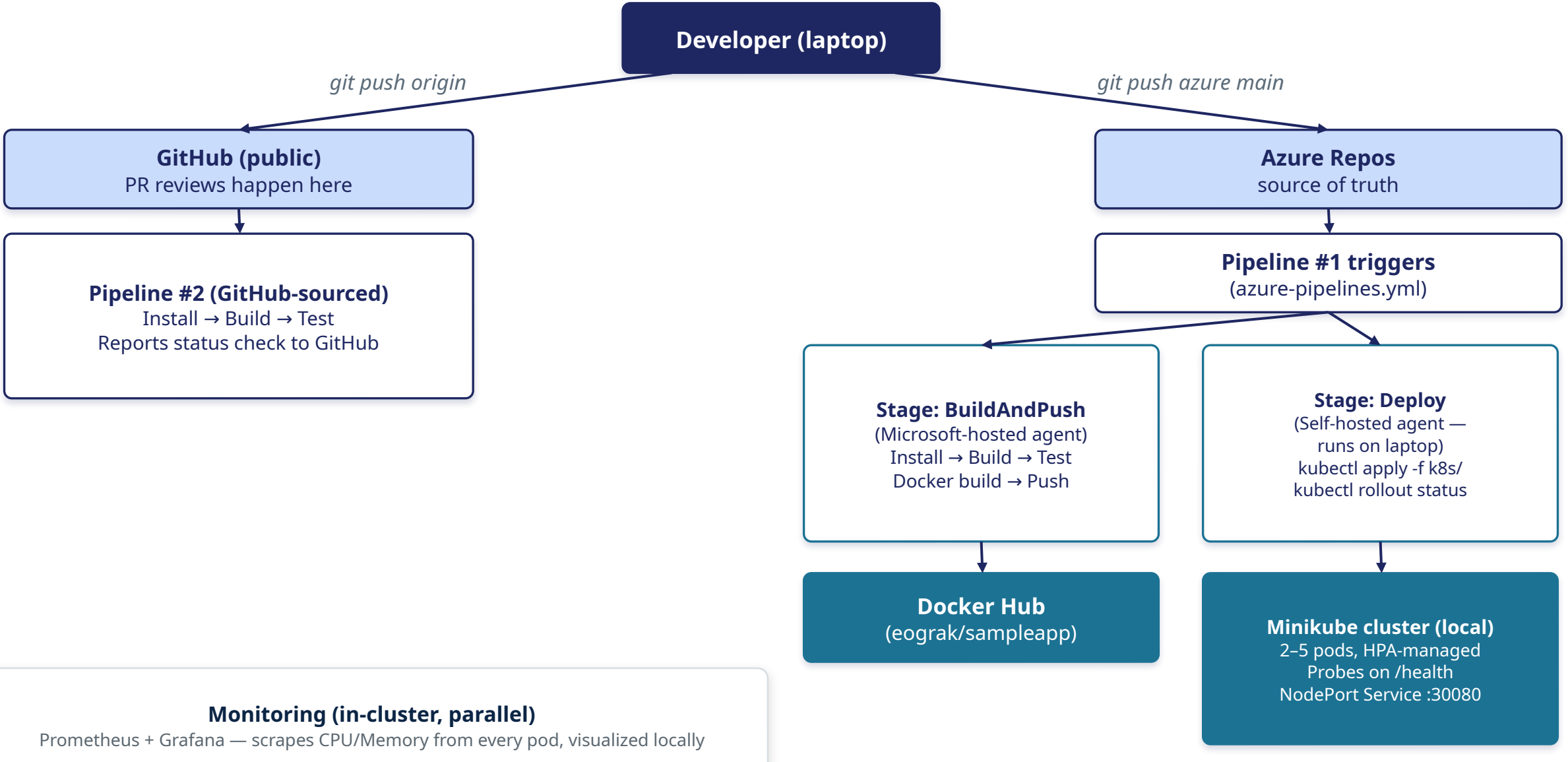
4

Challenges

What went wrong, and what it taught

CI/CD Pipeline Architecture

.NET application — Azure DevOps CI/CD to a Minikube cluster



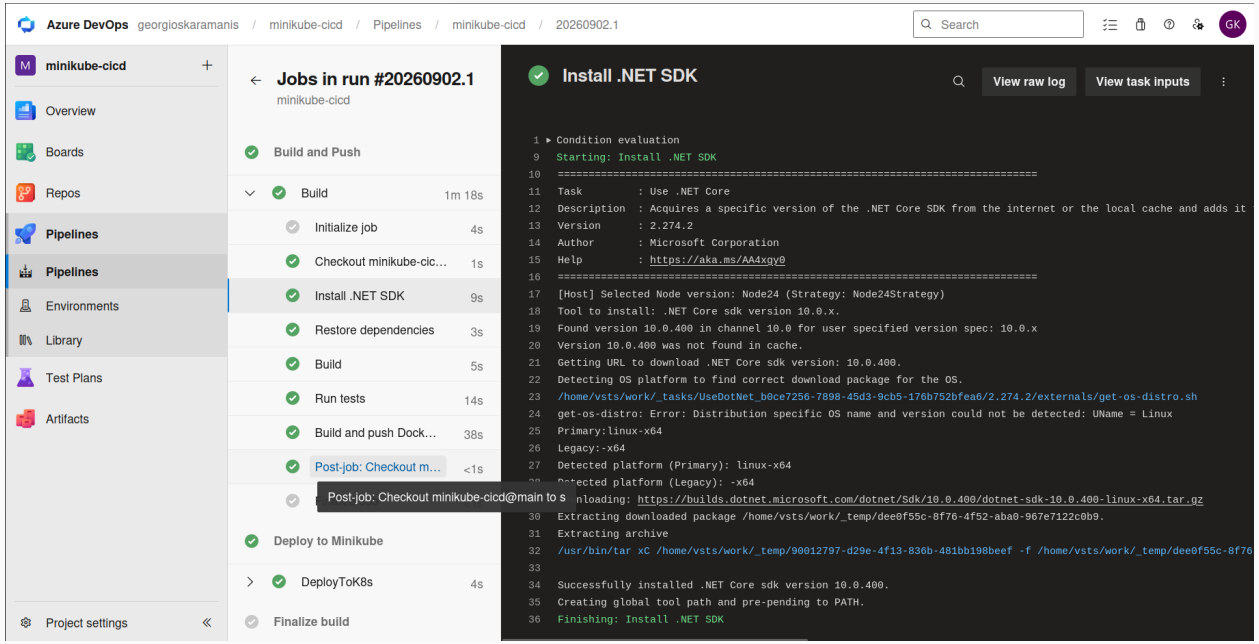
TASK 1

Azure DevOps Configuration

- Azure DevOps project + Git repo, mirrored to a public GitHub repo for visible commit history
- Minimal ASP.NET Core 10 Web API with /weatherforecast and a custom /health endpoint
- YAML build pipeline: triggers on every commit to main, installs SDK, restores, builds

WHY IT MATTERS

YAML pipelines version the CI/CD definition alongside the code. A feature-branch + PR workflow was kept consistent for the entire project, giving a clean, incremental history.



TASK 2

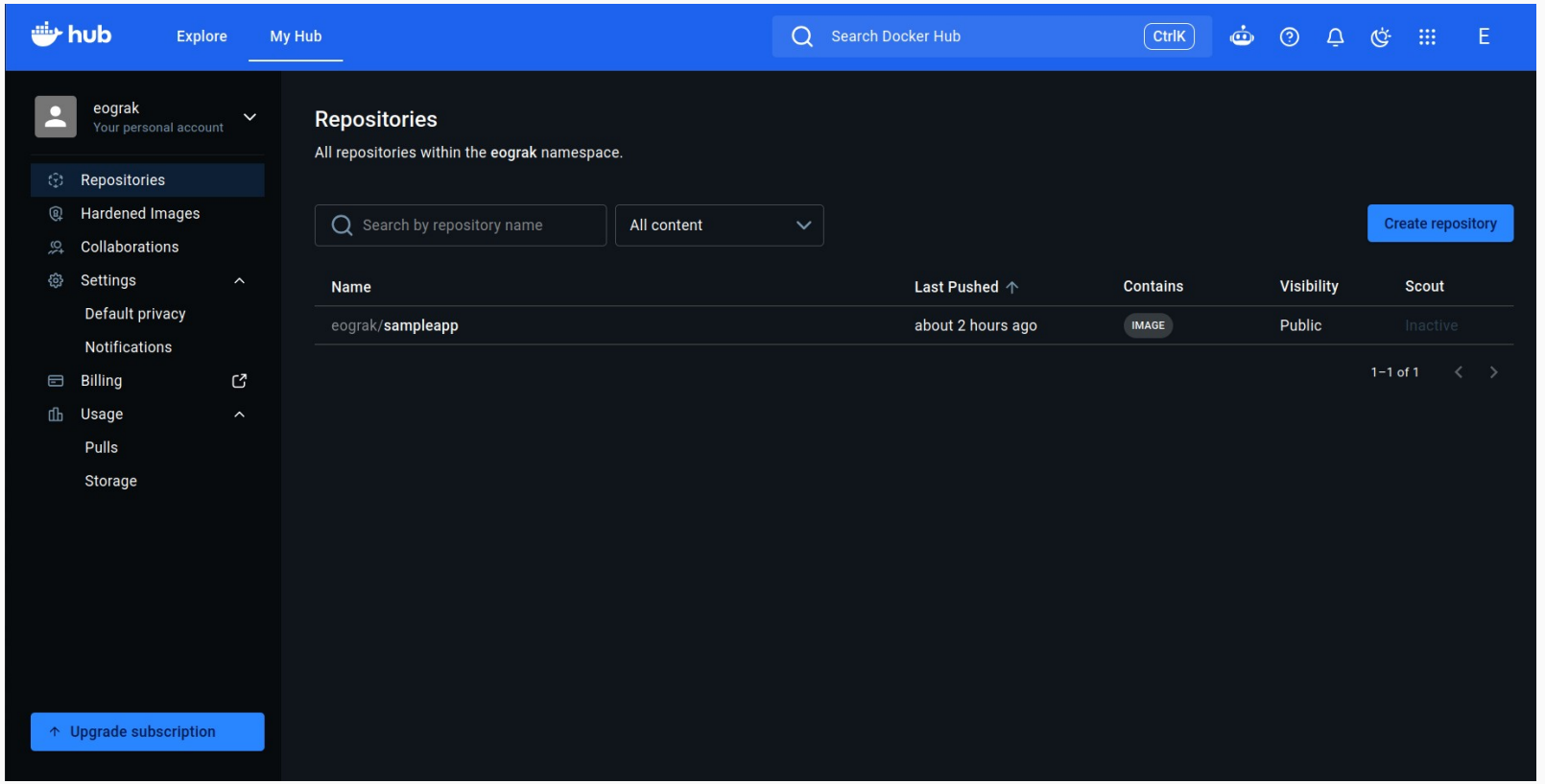
Docker Image Creation

Docker Image Multi stage build

Build stage
.NET 10 SDK image
restore · build · publish

→

Runtime stage
ASP.NET runtime image
copies published output only



INFO

- Multi-stage build → small final image, no SDK/compiler.
- Docker@2 task builds, tags with a unique build ID + latest, and pushes to Docker Hub via a secure service connection — no credentials in the YAML.

TASK 3

Kubernetes Deployment

Azure Pipelines
Microsoft-hosted
cloud agent

×

No network route

Minikube
Local cluster,
laptop-only

SOLUTION: SELF-HOSTED AGENT

A small agent process runs directly on the laptop and polls Azure DevOps for work. The pipeline's Deploy stage routes to this agent specifically, giving it the same local network access kubectl already has.

TWO-STAGE PIPELINE

BuildAndPush

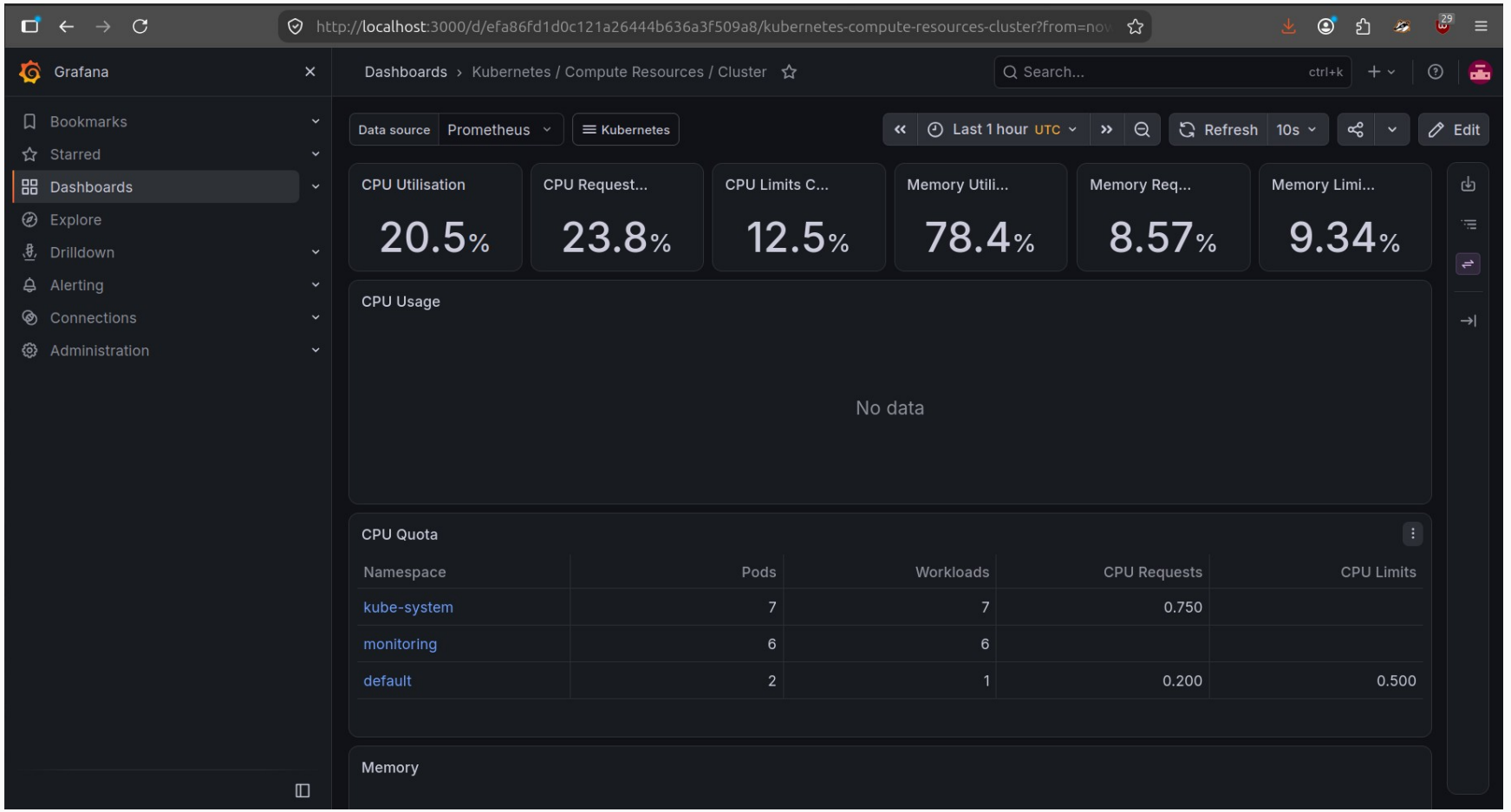
Microsoft-hosted agent

Deploy

Self-hosted agent — kubectl apply + rollout status

TASK 4

Monitoring – Grafana Dashboard



117%
Peak CPU observed under load

2 → 4
Pods scaled up automatically

KPI COVERAGE

CPU Saturation **Covered**

Memory Usage **Covered**

Error Rate (restart count) **Covered**

Availability **Covered**

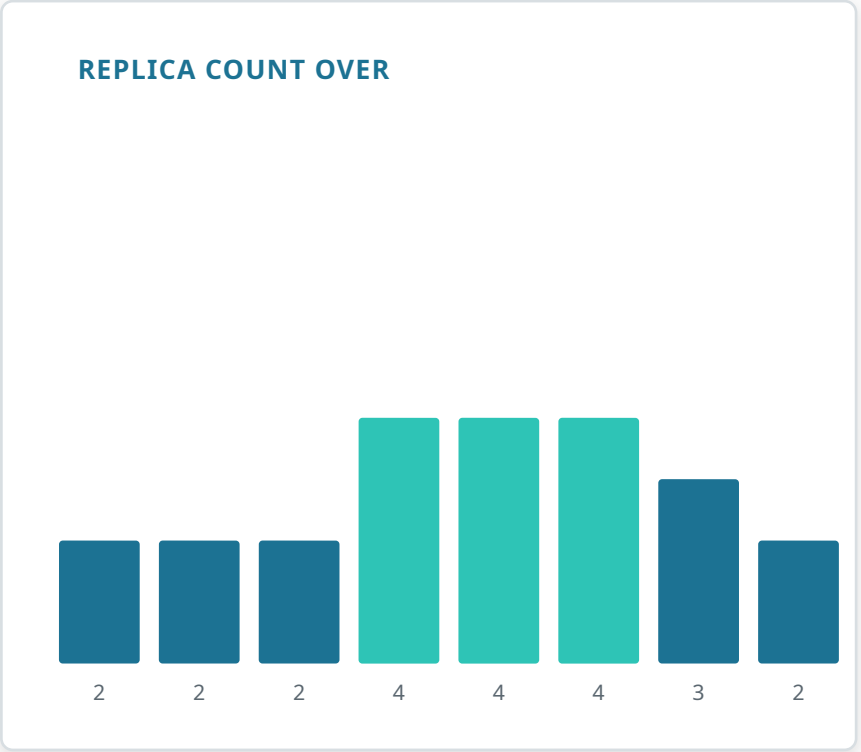
Latency / APM tracing **Known gap**

BONUS — HORIZONTAL POD AUTOSCALER

Verified under real, generated load

```
eog@eog-ThinkPad-X270-W10DG:~/GitHub/minikube-cicd$ kubectl get hpa --watch
NAME          REFERENCE                          TARGETS                        MINPODS   MAXPODS   REPLICAS   AGE
sampleapp-hpa   Deployment/sampleapp-deployment   cpu: 1%/70%, memory: 34%/80%   2         5         2          18m
sampleapp-hpa   Deployment/sampleapp-deployment   cpu: 22%/70%, memory: 38%/80%   2         5         2          18m
sampleapp-hpa   Deployment/sampleapp-deployment   cpu: 117%/70%, memory: 38%/80%   2         5         2          19m
sampleapp-hpa   Deployment/sampleapp-deployment   cpu: 117%/70%, memory: 38%/80%   2         5         4          19m
sampleapp-hpa   Deployment/sampleapp-deployment   cpu: 62%/70%, memory: 33%/80%    2         5         4          20m
sampleapp-hpa   Deployment/sampleapp-deployment   cpu: 58%/70%, memory: 33%/80%    2         5         4          21m
sampleapp-hpa   Deployment/sampleapp-deployment   cpu: 60%/70%, memory: 33%/80%    2         5         4          22m
sampleapp-hpa   Deployment/sampleapp-deployment   cpu: 53%/70%, memory: 33%/80%    2         5         4          23m
sampleapp-hpa   Deployment/sampleapp-deployment   cpu: 50%/70%, memory: 33%/80%    2         5         4          24m
sampleapp-hpa   Deployment/sampleapp-deployment   cpu: 44%/70%, memory: 33%/80%    2         5         4          25m
sampleapp-hpa   Deployment/sampleapp-deployment   cpu: 1%/70%, memory: 33%/80%     2         5         4          26m
sampleapp-hpa   Deployment/sampleapp-deployment   cpu: 1%/70%, memory: 33%/80%     2         5         4          27m
sampleapp-hpa   Deployment/sampleapp-deployment   cpu: 1%/70%, memory: 33%/80%     2         5         4          28m
sampleapp-hpa   Deployment/sampleapp-deployment   cpu: 1%/70%, memory: 33%/80%     2         5         4          29m
sampleapp-hpa   Deployment/sampleapp-deployment   cpu: 1%/70%, memory: 33%/80%     2         5         3          29m
sampleapp-hpa   Deployment/sampleapp-deployment   cpu: 1%/70%, memory: 33%/80%     2         5         3          30m
sampleapp-hpa   Deployment/sampleapp-deployment   cpu: 1%/70%, memory: 33%/80%     2         5         3          31m
sampleapp-hpa   Deployment/sampleapp-deployment   cpu: 1%/70%, memory: 36%/80%     2         5         2          31m
```

Terminal-kubectl: Replicas reacting to load generator



Scaled 2→4 within seconds of crossing target| Gradual step-down 4→3→2 after load stopped.

BONUS — PROBES & LAYERED TESTING

Two more safety mechanisms, both verified live

HEALTH PROBES

✓ **Startup**

1s interval, 30s grace — handles slow boots

✓ **Liveness**

10s interval, 3 fails → restart stuck pods

✓ **Readiness**

5s interval, 3 fails → pulled from traffic

TESTS RUN IN THREE PLACES

Local

dotnet test, on demand during development

Pre-merge

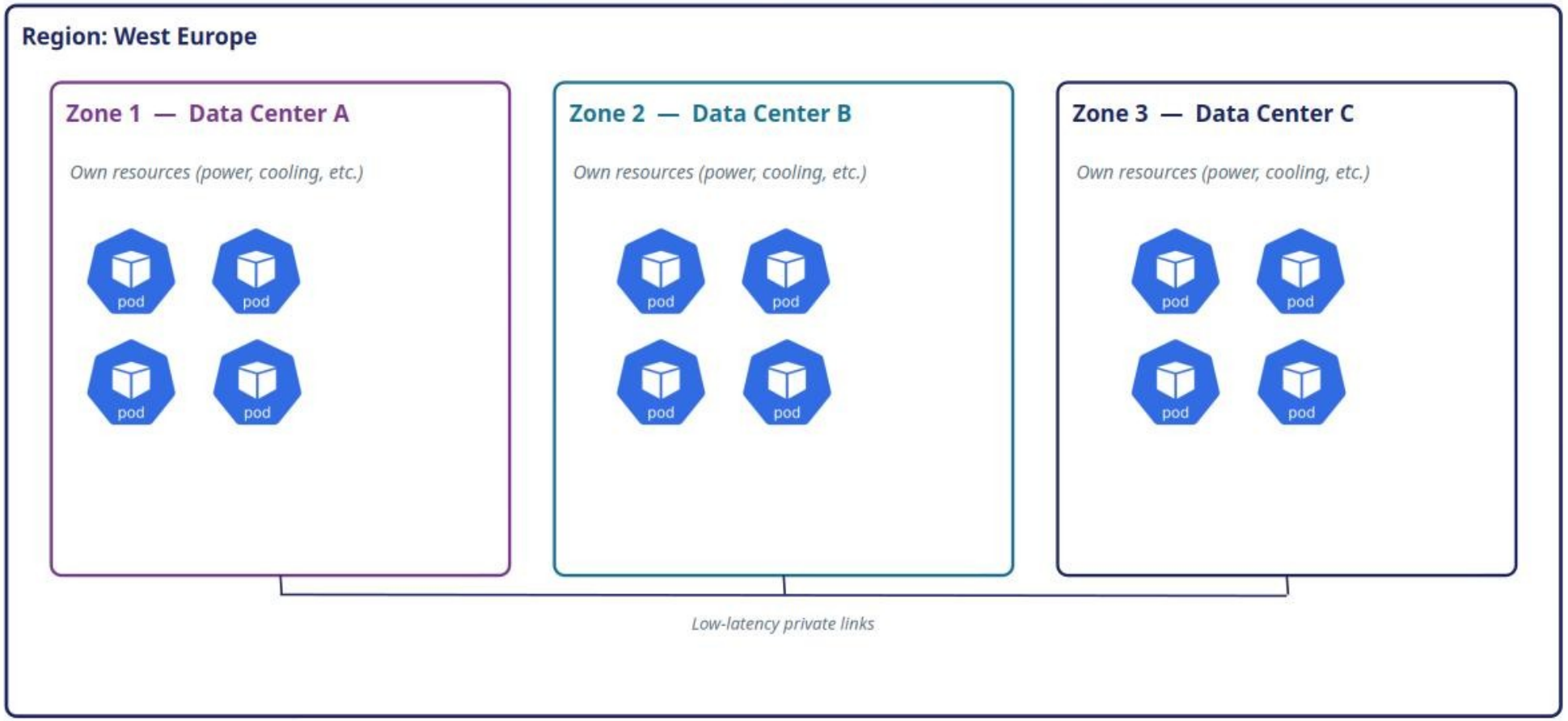
GitHub PR check — second pipeline, GitHub-sourced, blocks a bad merge

Post-merge

Main CD pipeline re-runs tests, blocks Docker build/deploy on failure

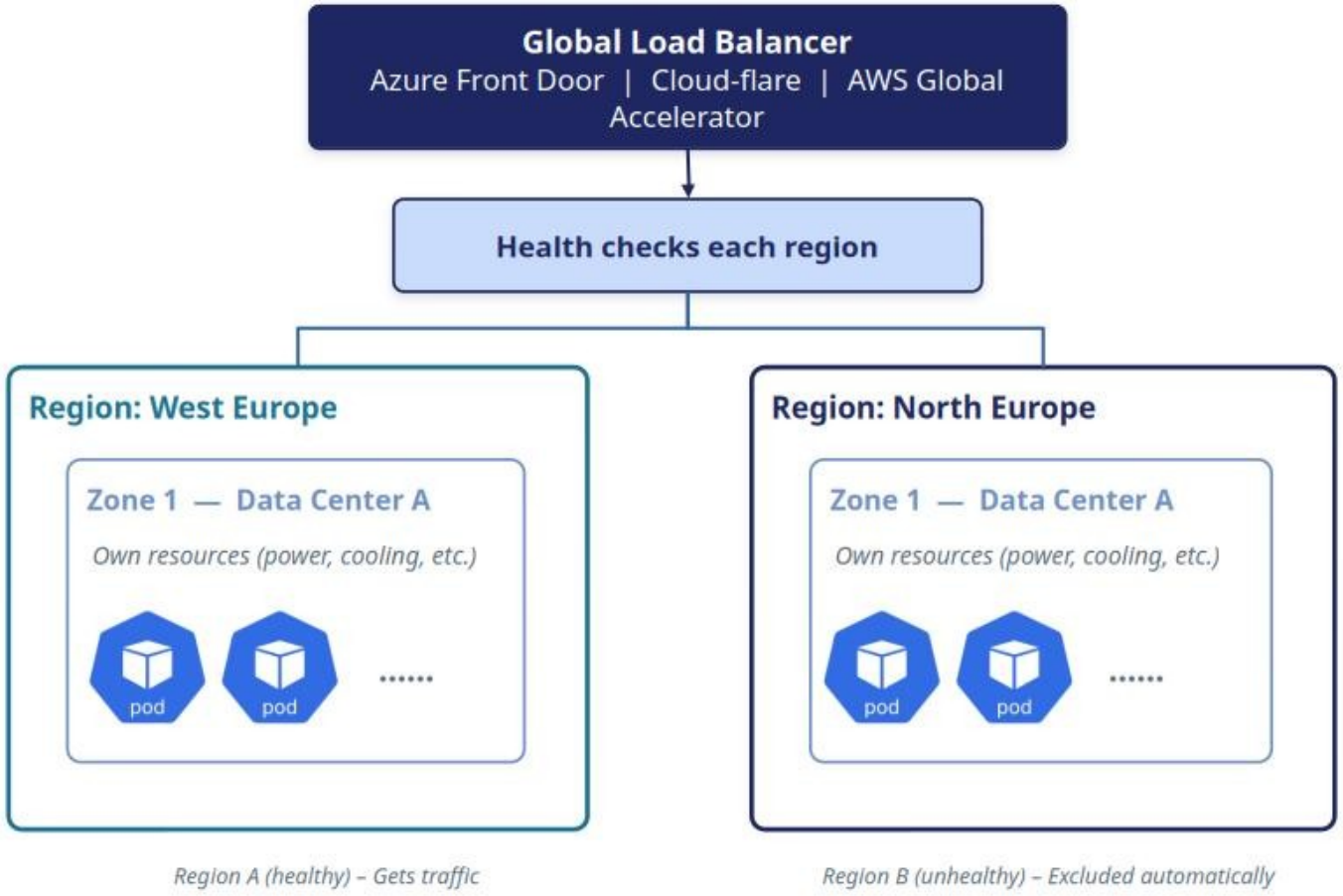
Multi-Zone Redundancy within a Region

Availability-zone isolation keeps pods running if one data center fails



Multi-Region Fail-over Strategy

Global traffic routing across independent Azure regions



CHALLENGES FACED

Real obstacles, resolved along the way

Docker's apt repo had no Ubuntu 26.04 build yet

Pointed at the 24.04 (noble) repo — fully compatible

.dockerignore omission broke the first Docker build

Local bin/obj artifacts were silently overwriting a fresh restore

Cloud agents can't reach a local-only cluster

Solved with a self-hosted agent running on the laptop

Program class wasn't visible to the test project

Standard fix: public partial class Program {} for WebApplicationFactory

Thank you

github.com/Ge0rak/minikube-cicd